

System Design Document: CodeMafia



1. Requirements Gathering

Functional Requirements (FRs)

The core features the system must perform.

1. **Lobby Management:** Users must be able to create, join, and leave multiplayer rooms using a unique 4-6 character room code.
2. **Role Assignment:** The system must randomly assign 1 Impostor and the rest as Civilians upon game start.
3. **Real-Time Collaborative Editing:** All players in a room must see code changes made by others in real-time.
4. **In-Browser Execution:** The system must evaluate the Python code against hidden test cases without sending the code to a backend server.
5. **Game State Synchronization:** The server must sync the current game phase (Lobby, Playing, Meeting, End) to all clients.
6. **Emergency Meetings & Voting:** Players can halt the coding phase to trigger a chat/voting screen. The system must tally votes and eliminate the chosen player.
7. **Win/Loss Conditions:** The system must evaluate test results (Civilian win) or timer expiration (Impostor win).

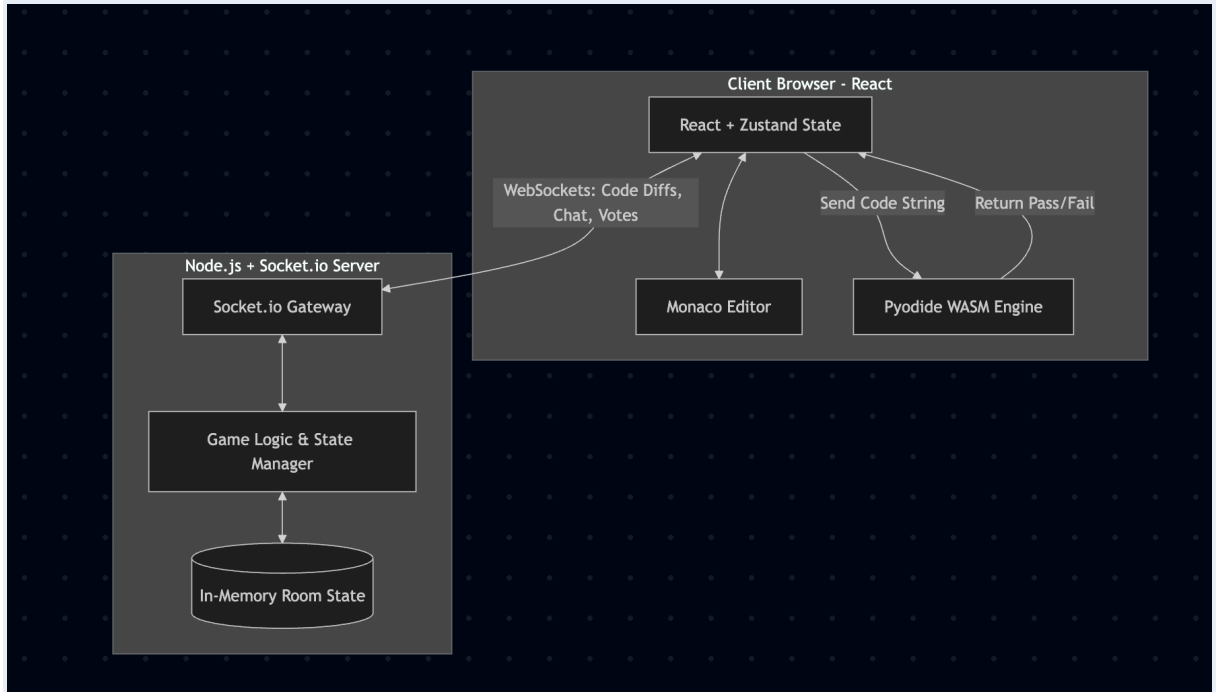
Non-Functional Requirements (NFRs)

How the system should behave (performance, security, etc.).

1. **Low Latency (< 100ms):** WebSocket communication must be near-instantaneous to prevent jarring cursor jumps or delayed code syncs.
2. **Security (Zero Server RCE):** User-submitted code must **never** run on the Node.js backend. All execution must happen locally in the client's WebAssembly sandbox.
3. **Server-Authoritative State:** Clients cannot be trusted to dictate who won or what phase the game is in. The server must hold the "source of truth."
4. **Scalability (Room-Based Partitioning):** The server logic must process events per-room, ensuring a game in Room A does not block the thread for Room B.

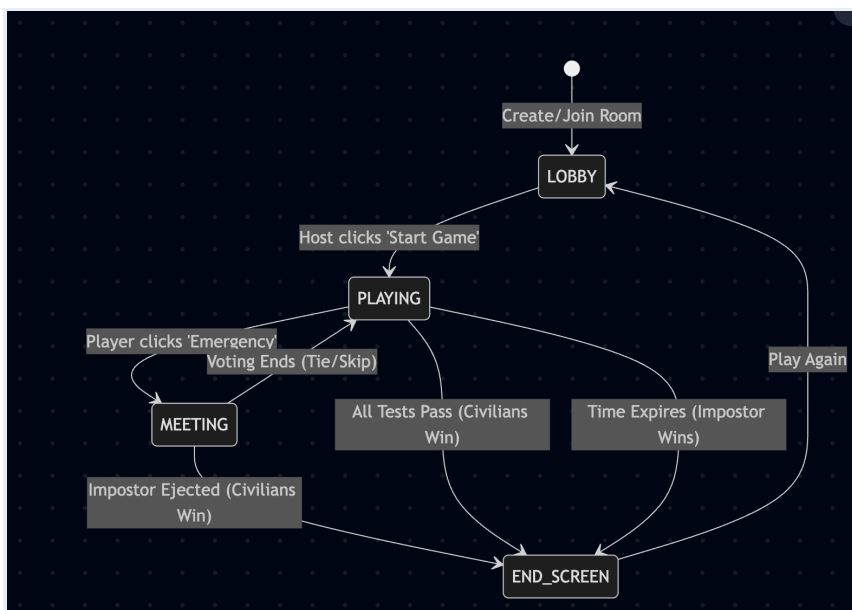
2. High-Level Architecture Diagram

CodeMafia uses a **Client-Server architecture with WebSockets**. The frontend handles the heavy lifting of code execution (via WASM) and UI, while a lightweight Node.js backend acts purely as a traffic cop, routing messages and maintaining the official game state.



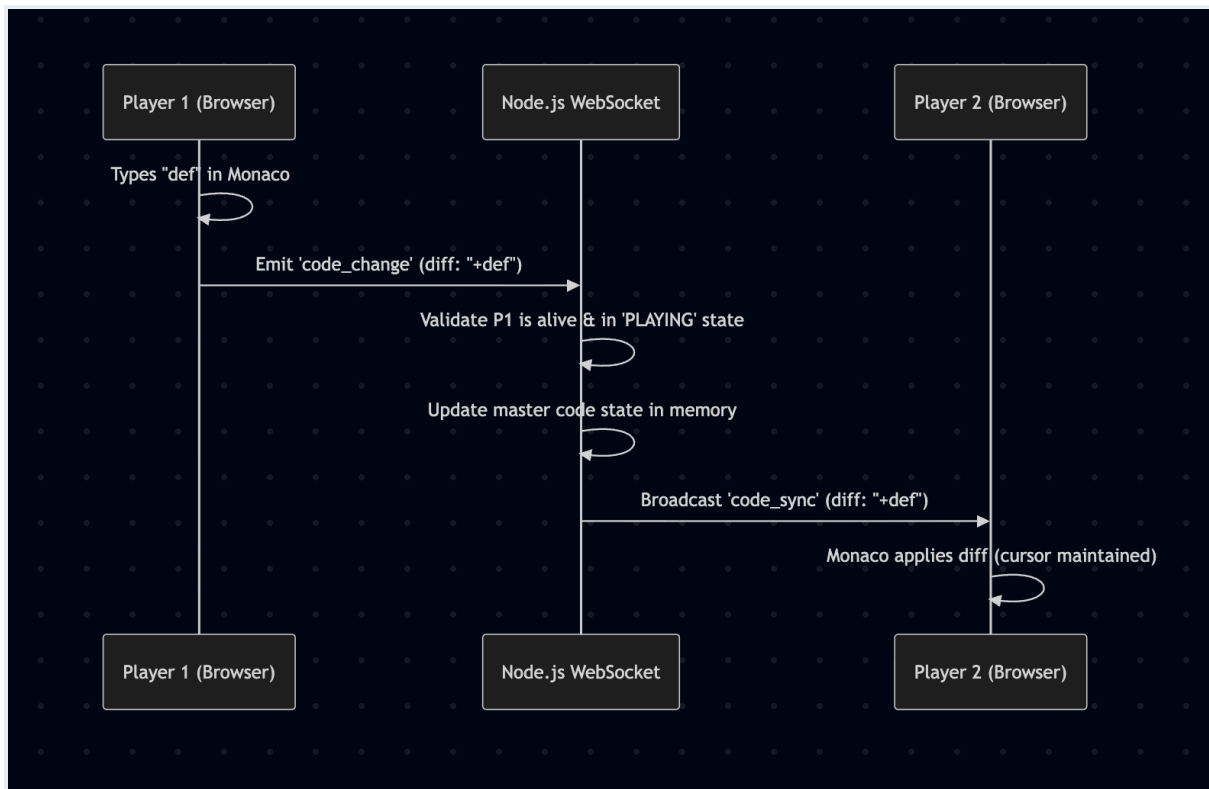
3. The Game State Machine (State Diagram)

The entire application relies on a finite state machine. The Node.js server controls these transitions and broadcasts them to the React frontend.



4. Real-Time Data Flow (Sequence Diagram: Collaborative Editing)

Handling collaborative text without messing up cursors is the hardest part. Here is how the data flows when a player types code.



5. Core Data Models (TypeScript / Server Memory)

Because this is a session-based game, you do not need a heavy database like PostgreSQL initially. You can store the active game state in the Node.js server's RAM (or Redis if scaling to multiple servers).

The Global Room State

- `interface GameRoom {`
- `roomId: string;`
- `hostId: string;`
- `phase: 'LOBBY' | 'PLAYING' | 'MEETING' | 'END_SCREEN';`
- `players: Record<string, Player>; // Map of socket IDs to Player objects`
- `currentCode: string; // The master string of the code editor`
- `problemId: string; // The current DSA problem being solved`
- `timerSeconds: number; // Remaining time`
- `meetingData?: MeetingState; // Only exists during meetings`
- `}`

The Player Object

- `interface Player {`
- `socketId: string;`
- `username: string;`
- `role: 'CIVILIAN' | 'IMPOSTOR' | 'UNASSIGNED';`
- `isAlive: boolean;`
- `color: string; // For UI (e.g., Red, Blue, Green)`
- `}`

6. Socket.io Event API Design

To keep the network clean, you must standardize the events sent between the Client and Server.

Client-to-Server Events (Actions)

- `join_room (username, roomId)` -> Connects the user to a socket room.
- `start_game ()` -> Host triggers game start. Server assigns roles.
- `code_update (newCode)` -> Sends text editor changes.
- `run_tests ()` -> Client notifies server they are running tests.
- `tests_passed ()` -> Client proves they solved the challenge. (Server ends game).
- `call_meeting ()` -> Halts the game and switches phase to MEETING.
- `cast_vote (targetPlayerId)` -> Registers a vote during a meeting.

Server-to-Client Events (Broadcasts)

- `room_state_update` (GameRoom object) -> Sent whenever someone joins/leaves or phase changes.
- `role_reveal` (role) -> Sent privately to each socket on game start.
- `code_synced` (newCode) -> Broadcasts code changes to everyone *except* the sender.
- `meeting_started` (callerId) -> Forces all clients to show the voting UI.
- `chat_message_received` (msg, author) -> Syncs chat during meetings.
- `game_over` (winner, reason) -> Triggers the victory/defeat animations.

7. Security & Anti-Cheat Measures

1. **Code Execution Isolation:** By using Pyodide, if an Impostor writes `os.system("rm -rf /")`, it only executes in their own browser tab's WebAssembly sandbox. It cannot crash the server or harm other players.
2. **Secret Roles:** The `role` property is **never** sent in the public `room_state_update` payload. Roles are sent directly to individual socket IDs via `socket.emit('role_reveal')`. Otherwise, players could open browser DevTools and see who the Impostor is.
3. **Action Validation:** The server must check if a player `isAlive === true` before accepting `code_update` or `cast_vote` events from them.